

150 Interview Questions

Fraud Detection ML Project

Class Imbalance · Models · Metrics · FastAPI · Docker · CI/CD · MLflow · SHAP

[Easy] basics | [Medium] project depth | [Hard] senior/system design

Category 1 — Class Imbalance (Q1–Q20)

Q1. What is class imbalance and why is it a problem?

[Easy]

Class imbalance: one class dominates. Fraud = 0.17%, legit = 99.83%. Models learn to predict majority always. Gets 99.83% accuracy, 0% fraud recall. Accuracy is a lie. Use Recall, Precision, AUC-PR.

Q2. Why is accuracy useless for fraud detection?

[Easy]

A model predicting 'not fraud' always gets 99.83% accuracy. It catches zero fraud. Accuracy = (TP+TN)/all. When TN is massive (284k legit), high accuracy means nothing about fraud performance.

Q3. What is SMOTE? How does it differ from random oversampling?

[Medium]

SMOTE: picks minority sample, finds k nearest minority neighbours, places new synthetic sample between them (interpolation). Random oversampling: just copies existing samples → overfitting. SMOTE creates genuinely new samples in the feature region.

Q4. Must SMOTE be applied before or after train/test split?

[Medium]

ALWAYS after split, ONLY on training data. Before split = synthetic samples from test data appear in training = data leakage = dishonest evaluation. Test set must stay real data.

Q5. What does k_neighbors=5 control in SMOTE?

[Medium]

How many nearest fraud neighbours to consider when generating a new synthetic sample. Higher k = new samples spread wider (more diversity). Lower k = tighter clusters around existing samples.

Q6. What is scale_pos_weight in XGBoost?

[Medium]

Multiplies gradient contribution of positive (fraud) samples. Calculation: count(legit) / count(fraud) = 284315/492 = 578. Each fraud sample contributes 578x more to tree learning.

Q7. Can you use SMOTE and scale_pos_weight together?

[Hard]

No. SMOTE already balances classes. scale_pos_weight then further upweights the now-balanced fraud class. Double-correction: model becomes too aggressive, precision drops severely.

Q8. What is class_weight='balanced' in scikit-learn?

[Medium]

Auto-computes: $\text{weight}_i = n_{\text{samples}} / (n_{\text{classes}} * n_{\text{samples}_i})$. Fraud weight ≈ 578 . Adjusts loss function — fraud mistakes cost 578x more. One-liner fix without synthetic data.

Q9. What is ADASYN vs SMOTE?

[Hard]

ADASYN (Adaptive Synthetic Sampling): generates more samples where classification is difficult (boundary regions). SMOTE generates uniformly. ADASYN focuses on hard-to-learn areas but may amplify noise.

Q10. What is the imbalance ratio and when should you worry?

[Medium]

Ratio = majority / minority. 578:1 in our dataset. Start correcting at 10:1. At 100:1 most models fail without handling. At 1000:1+ consider anomaly detection approaches.

Q11. Explain the Precision-Recall tradeoff.

[Easy]

Precision: of fraud alerts, how many are real. Recall: of actual frauds, how many caught. Lower threshold → higher recall, lower precision. This tradeoff is tuned via classification threshold.

Q12. What is the F2-beta score? When use it over F1?

[Medium]

$F2 = (1+4) \cdot P \cdot R / (4 \cdot P + R)$. Weights recall 2x more than precision. Use when FN (missed fraud) costs more than FP (false alarm). Use F0.5 when precision matters more.

Q13. What is AUC-PR and why better than AUC-ROC for fraud?

[Hard]

AUC-PR = area under Precision-Recall curve. AUC-ROC uses $FPR = FP / (FP + TN)$. With 284k TN, FPR looks tiny even for a bad model. AUC-PR ignores TN — honestly measures rare class performance.

Q14. What is the no-skill baseline for a PR curve?

[Hard]

Horizontal line at $y = \text{fraud rate} = 0.17\%$. Model must be significantly above this line. On ROC, no-skill = diagonal (AUC=0.5) regardless of class distribution — looks better for imbalanced data.

Q15. What are all four confusion matrix cells for fraud?

[Easy]

TP: fraud caught (money saved). FP: legit blocked (angry customer). FN: fraud missed (money lost — worst). TN: legit passed (normal). Minimise FN. Manage FP vs customer experience.

Q16. What is stratify=y in train_test_split?

[Medium]

Ensures both sets maintain the same fraud ratio (0.17%). Without: random chance might put all fraud in one set. Evaluation on zero fraud = meaningless.

Q17. What alternatives exist to SMOTE?

[Hard]

Undersampling: Random, Tomek Links. Oversampling: ADASYN, Borderline-SMOTE. Algorithmic: `class_weight`, `scale_pos_weight`. Ensemble: `BalancedBaggingClassifier`, `EasyEnsemble`. Hybrid: SMOTE + Tomek Links often best.

Q18. What is cost-sensitive learning?

[Hard]

Directly assign misclassification costs in the loss function. $\text{sample_weight} = y_{\text{train}} \cdot 577 + 1$ (fraud=578x weight, legit=1x). More flexible than `class_weight`: can vary weights by transaction amount.

Q19. What is the Matthews Correlation Coefficient?

[Hard]

$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$. Range: -1 to +1. Considers all four confusion matrix cells. Most informative single metric for imbalanced data.

Q20. How do you present model results to a non-technical stakeholder?

[Medium]

Never say F1 or AUC. Say: 'Of 100 fraud cases, we catch 89. 11 slip through costing £122 each = £1,342 lost per 100 fraud events. We block 15 legit customers needing a support call. Net: we save £5,800 per 100 fraud events vs no model.'

Category 2 — Machine Learning Models (Q21–Q45)

Q21. Why build a Logistic Regression baseline first?

[Easy]

Establishes performance floor. If XGBoost only beats LR by 1%, complexity not justified. Also verifies pipeline is correct before investing in complex models.

Q22. What is gradient boosting in plain English?

[Easy]

Builds trees sequentially. Each tree corrects the previous tree's mistakes. Final prediction = sum of all trees $\times$ learning_rate. Each tree is weak; combination is strong.

Q23. What does n_estimators control in XGBoost?

[Easy]

Number of trees to build. More = potentially higher accuracy but risk of overfitting and slower inference. With early_stopping_rounds, set high (1000) and let XGBoost find optimal automatically.

Q24. What is max_depth and how does it affect overfitting?

[Medium]

Limits tree depth. Depth 6 = max 64 leaves per tree. Deeper = more complex patterns but higher overfitting risk. Typical fraud: depth 4-8. Shallower = faster + more regularised.

Q25. Explain learning_rate (eta). Why 0.05?

[Medium]

Scales each tree's contribution. Lower lr = smaller steps = better generalisation but needs more trees. 0.05 + 300 trees is proven. Lower lr (0.01) = increase n_estimators to 1500+.

Q26. What is subsample in XGBoost?

[Medium]

Fraction of rows randomly selected for each tree. subsample=0.8: each tree trains on 80% of rows. Row-level bagging = regularisation, reduces variance.

Q27. What is colsample_bytree?

[Medium]

Fraction of features randomly selected for each tree. Column-level bagging. Like Random Forest. Prevents one feature from dominating, adds diversity.

Q28. What is early stopping in XGBoost?

[Medium]

Monitors validation metric. Stops when it doesn't improve for N rounds. Set n_estimators=1000, early_stopping_rounds=50. Prevents overfitting, auto-finds optimal n_estimators.

Q29. What is the difference between XGBoost and Random Forest?

[Medium]

RF: trees in parallel, bootstrap sampling, average predictions. XGBoost: sequential, each corrects previous. RF: faster, more robust. XGBoost: higher accuracy with tuning. XGBoost has scale_pos_weight advantage for fraud.

Q30. What is LightGBM vs XGBoost?

[Hard]

LightGBM: leaf-wise growth (best leaf first) vs XGBoost level-wise. Histogram-based splits = 10x faster on large datasets. Both similar accuracy. LightGBM preferred for 10M+ rows.

Q31. What is regularisation? Types in XGBoost?

[Hard]

Prevents overfitting by penalising complexity. XGBoost: L1 (reg_alpha, drives weights to zero), L2 (reg_lambda, shrinks weights), min_child_weight (min samples per leaf), gamma (min gain for split).

Q32. What is cross-validation? Why StratifiedKFold?

[Medium]

CV: train/evaluate k times on different folds. More reliable than single split. StratifiedKFold: each fold maintains fraud ratio. Without stratify, some folds may have zero fraud.

Q33. What does high CV standard deviation indicate?

[Hard]

Model is sensitive to which data it trains on. Causes: overfitting, small positive class, data ordering. Fix: add regularisation, ensure stratification, shuffle data.

Q34. What is the bias-variance tradeoff?

[Hard]

Bias: systematic error from oversimplistic model (LR on complex patterns = underfitting). Variance: error from oversensitivity to training data (deep trees = overfitting). Regularisation controls variance.

Q35. What is data leakage? Give examples in this project.

[Hard]

Leakage: test/future info influences training. Examples: SMOTE before split, correlation on full dataset, using current scaler stats on test data. Makes metrics look great in dev, fail in production.

Q36. Difference between predict() and predict_proba()?

[Easy]

predict(): hard labels (0 or 1) using default threshold 0.5. predict_proba(): probability array (n_samples, 2). [:, 1] = fraud probability. Always use predict_proba for threshold tuning.

Q37. What is feature importance? Types in XGBoost?

[Hard]

weight: times feature used in splits. gain: average accuracy improvement when feature used (most informative). cover: average sample coverage. SHAP values give fairer importance than built-in.

Q38. When would LR beat XGBoost?

[Hard]

Very small dataset (<5k samples), strong linear relationships, constrained hardware (LR inference = one dot product), strict interpretability required, need results in seconds.

Q39. What is eval_metric='aucpr'?

[Medium]

XGBoost monitors AUC-PR on eval_set after each round. Used with early stopping. More appropriate than 'auc' (ROC) for imbalanced fraud data.

Q40. What is overfitting? How to detect it here?

[Medium]

Overfitting: great on train, poor on test. Detect: train AUC-PR >> test AUC-PR (gap > 0.05 suspicious). Also: high CV std. Fix: regularisation, reduce max_depth, early stopping.

Q41. What is ensemble learning?

[Hard]

Combining multiple models. Bagging (RF, parallel), Boosting (XGBoost, sequential), Stacking (meta-model on base predictions). Stacking LR+XGBoost often beats either alone.

Q42. What is concept drift?

[Hard]

Fraud patterns change as fraudsters adopt new techniques. Model trained in January may miss June's new attacks. Detect with KS tests on feature distributions. Trigger retraining when recall drops below threshold.

Q43. What is normalisation vs standardisation?

[Medium]

Normalisation (MinMax): scales to [0,1]. Standardisation (StandardScaler): mean=0, std=1. For fraud Amount (skewed with outliers): StandardScaler better. Tree models don't need either.

Q44. Why don't tree models need feature scaling?

[Medium]

Trees split by 'feature > threshold'. Relative ordering doesn't change with scaling. LR, SVM, KNN, NNs need scaling (distance/gradient-based).

Q45. What is a scikit-learn Pipeline?

[Hard]

Chains preprocessing + model: Pipeline([('scaler', StandardScaler()), ('model', XGBClassifier())]). Prevents leakage (scaler fits only on training fold in CV), enables clean deployment.

Category 3 — FastAPI & Production Deployment (Q46–Q80)

Q46. What is FastAPI vs Flask?

[Easy]

FastAPI: automatic Pydantic validation, auto OpenAPI docs, async support, faster. Flask: older, more tutorials. For ML APIs needing type safety and docs: FastAPI is standard.

Q47. What is Pydantic? How does it protect the API?

[Medium]

Data validation library. BaseModel subclass auto-validates incoming JSON. Wrong types or missing fields return 422 error before model code runs. Prevents bad data reaching inference.

Q48. Why load model at startup not per request?

[Medium]

Per-request load = 200-500ms disk read every call. Payment SLA = <100ms total. Memory access = microseconds. Lifespan pattern loads once into RAM. All requests share that in-memory object.

Q49. What is the lifespan pattern?

[Medium]

asynccontextmanager function. Code before yield runs at startup (load model). Code after yield runs at shutdown (cleanup). Replaces deprecated `@app.on_event('startup')`.

Q50. Difference between sync def and async def in FastAPI?

[Hard]

sync def: runs in thread pool — correct for CPU-bound inference. async def: runs in event loop — correct for I/O-bound work (DB, HTTP calls). Never do CPU work in async def — blocks event loop.

Q51. What is Docker and why containerise?

[Easy]

Docker packages app + dependencies + runtime into one image. Runs identically everywhere — eliminates 'works on my machine'. Guarantees same Python version, library versions, model files in production.

Q52. Explain Docker layer caching trick.

[Medium]

COPY requirements.txt → RUN pip install → COPY code. If only code changes, pip install layer is cached and reused. Reverse order forces full pip reinstall on every code change — slow.

Q53. COPY vs ADD in Dockerfile?

[Medium]

COPY: simply copies files. ADD: also downloads URLs and extracts tar archives. Always use COPY for ML deployment — explicit and predictable. ADD's extras cause surprising behaviour.

Q54. What is EXPOSE in Dockerfile?

[Easy]

Documentation only — records which port the app listens on. Does NOT open the port. Actual port mapping at runtime: `docker run -p 8000:8000`.

Q55. Why run as non-root user in Docker?

[Medium]

Root inside container = security vulnerability. If attacker escapes container, they have root on host. Create dedicated user: `RUN useradd -m appuser / USER appuser`.

Q56. What is a health check endpoint?

[Medium]

GET /health returns {status:'ok'}. Kubernetes, load balancers poll it. If 3 consecutive failures, container restarted or traffic redirected. Without it: broken instances still receive traffic.

Q57. What is CORS? When do you need it for the fraud API?

[Hard]

Cross-Origin Resource Sharing: browser security mechanism. Server-to-server calls (payment gateway to fraud API): CORS not needed. Browser-based dashboard calling API: need CORSMiddleware with allowed origins.

Q58. What is uvicorn vs gunicorn?

[Medium]

Uvicorn: ASGI server, runs FastAPI, single process by default. Gunicorn: process manager. Together: gunicorn -w 4 -k uvicorn.workers.UvicornWorker — 4 worker processes on all CPU cores.

Q59. What HTTP status codes should fraud API return?

[Medium]

200: success. 422: invalid request (Pydantic auto-returns this). 500: unexpected model error. 503: model not loaded yet. 429: rate limit exceeded. 401: missing auth.

Q60. What is model versioning?

[Hard]

Track model versions in MLflow registry. URL versioning: /v1/predict, /v2/predict. Docker tags: fraud-api:v1, fraud-api:v2. Deploy v2 alongside v1, shift traffic gradually. Rollback = repoint to old tag.

Q61. What is blue-green deployment?

[Hard]

Run two environments: blue (current production) and green (new version). Switch load balancer from blue to green when green is validated. Instant rollback: switch back. Zero-downtime deployment.

Q62. What is canary deployment?

[Hard]

Route small % (e.g. 5%) of traffic to new model. Monitor metrics on that slice. If metrics hold, increase to 25%, 50%, 100%. Zero risk — majority of users still on stable version during validation.

Q63. What is shadow mode deployment?

[Hard]

New model receives all traffic, makes predictions, but decisions NOT used. Only old model enforces blocks. New model's predictions logged for comparison after ground truth arrives. Zero risk validation.

Q64. How to handle model updates without downtime?

[Hard]

Rolling update (Kubernetes), Blue-green, Canary. Always: shadow test first, monitor 24h, keep old model deployable for rollback. Use MLflow registry stage transitions.

Q65. What monitoring would you set up?

[Hard]

Operational: latency p50/p95/p99, error rate, throughput. Model: recall on confirmed fraud (delayed ground truth), false positive rate, score distribution. Drift: KS tests on features weekly. Alerting: if recall < 80%.

Q66. What is latency vs throughput?

[Medium]

Latency: time to process one request. Throughput: requests/second. Fraud API target: <100ms latency. At 1M tx/day = 12 RPS average, 100+ RPS burst. Plan for 10x peak capacity.

Q67. What is a microservices architecture?

[Hard]

App split into independent services communicating via APIs. Payment System: Transaction Service, User Service, Fraud API, Notification Service. Each scales independently. Fraud API updated without redeploying payment system.

Q68. What is requirements.txt? Why pin versions?

[Easy]

Lists all dependencies with exact versions: `xgboost==2.0.3`. Without pinning: pip installs latest version which may have breaking changes. Pinned versions ensure identical packages across all environments.

Q69. What is GitHub Actions?

[Easy]

CI/CD platform built into GitHub. YAML workflow files in `.github/workflows/` trigger on push/PR. Jobs run on GitHub-hosted Ubuntu VMs. Free for public repos. Shows green checkmark on repo.

Q70. What is a GitHub Actions secret?

[Medium]

Encrypted variable stored in GitHub Settings → Secrets. Referenced as `${{ secrets.DOCKER_PASSWORD }}` in YAML. Never hardcode credentials in workflow files — visible in repo history.

Q71. What is the .gitignore file?

[Easy]

Lists files/patterns git should not track. Never commit: `*.pkl` (model files), `*.csv` (training data), `.env` (credentials), `__pycache__` (compiled Python). Large files make repo slow and may expose sensitive data.

Q72. Why use Docker Hub?

[Medium]

Public container registry. Store Docker images. Teams pull them. Free tier for public repos. Alternatives: AWS ECR, Google Artifact Registry, GitHub Container Registry. `docker push` / `docker pull`.

Q73. What is Render.com for deployment?

[Easy]

Free cloud hosting for Docker containers. Connect GitHub repo, auto-deploy on push. Free tier: sleeps after 15min inactivity. Good for portfolio demos. Paid tier for production use.

Q74. What is Kubernetes?

[Hard]

Container orchestration platform. Manages: scheduling (which server to run on), scaling (auto-add pods under load), health monitoring (restart failed pods), rolling updates. Production fraud APIs run 3-10 replicas.

Q75. What is a REST API?

[Easy]

HTTP-based API using standard methods. GET: retrieve data. POST: send data for processing. Your fraud API: `POST /predict` (get fraud decision), `GET /health` (server status), `GET /docs` (auto-documentation).

Q76. What is JSON?

[Easy]

JavaScript Object Notation. Text format for structured data. Language-agnostic. FastAPI auto-serialises Python dicts to JSON for responses, deserialises JSON requests to Python objects.

Q77. How to stress test the fraud API?

[Hard]

Use locust: define POST /predict with realistic payloads, ramp 100 to 1000 concurrent users. Measure p50/p95/p99 latency and error rate. Find: at what concurrency does latency exceed 100ms.

Q78. What is a feature store?

[Hard]

Centralised repository for ML features. Offline store: historical features for training. Online store: latest features per entity (customer, card), served <10ms. Tools: Feast, Tecton, AWS SageMaker Feature Store.

Q79. What is batch vs real-time inference?

[Medium]

Real-time: one transaction, immediate response (<100ms). Used for online payments. Batch: millions of transactions scored overnight. Used for: retroactive analysis, model monitoring, retraining data prep.

Q80. What is A/B testing for ML?

[Hard]

Route X% traffic to model A, rest to model B. Compare: fraud catch rate, false positive rate, approval rate on live traffic. Challenge for fraud: labels arrive 30-90 days later (chargebacks). Need weeks of data.

Category 4 — MLflow, SHAP, CI/CD & System Design (Q81–Q120)

Q81. What is MLflow?

[Easy]

MLOps platform: experiment tracking (log params/metrics/artifacts), model registry (version lifecycle), project reproducibility, model serving. Without it: lose track of which hyperparameters produced which results.

Q82. What is an MLflow run?

[Medium]

One execution of training code. Has: run_id, params (n_estimators=300), metrics (auc_pr=0.89), artifacts (model.pkl, plots), tags. Grouped under experiments.

Q83. Difference between log_param and log_metric?

[Medium]

log_param: hyperparameter set before training (n_estimators). Immutable for the run. log_metric: result measured after training (recall=0.87). Can be logged multiple times (per-epoch time series).

Q84. What is the MLflow model registry?

[Hard]

Central model store with lifecycle stages: None → Staging → Production → Archived. Load by name: mlflow.xgboost.load_model('models:/fraud-detector/Production'). Enables auditing and safe rollback.

Q85. How to compare multiple MLflow runs?

[Medium]

MLflow UI: select runs, click Compare. Parallel coordinates plot shows param vs metric relationships. Programmatically: MlflowClient().search_runs(experiment_ids=['1']) returns DataFrame.

Q86. What is SHAP?

[Medium]

SHapley Additive exPlanations. Game-theory-based method to explain individual predictions. SHAP value for feature F = how much F pushed prediction above/below average. Created by Scott Lundberg (2017).

Q87. What is a SHAP value intuitively?

[Medium]

Baseline = average model output. SHAP value = how much this feature pushed this prediction above/below baseline. Positive = pushed toward fraud. Negative = pushed toward legitimate.

Q88. What is the SHAP summary (beeswarm) plot?

[Medium]

Each dot = one transaction. X-axis = SHAP value (right = toward fraud). Colour = feature value (red=high, blue=low). Red dots on right = high feature value pushes toward fraud.

Q89. What is a SHAP waterfall plot?

[Medium]

Local explanation for one transaction. Starts at baseline. Each bar = one feature's contribution. Shows exactly why THIS transaction was flagged. Fraud analyst reads: 'V14 unusual, Amount 7x average, 3am transaction.'

Q90. TreeExplainer vs KernelExplainer?

[Medium]

TreeExplainer: fast, exact, tree-specific (XGBoost/RF). KernelExplainer: model-agnostic, uses sampling, much slower. Always use TreeExplainer for XGBoost.

Q91. Why is SHAP required for fraud detection compliance?

[Hard]

GDPR Article 22: right not to be subject to solely automated decisions. A declined payment is significant. Banks must explain the logic. SHAP provides auditable, feature-level explanation per transaction.

Q92. What is LIME vs SHAP?

[Hard]

LIME: fits local linear model around data point using perturbations. SHAP: Shapley values with theoretical guarantees (consistency, local accuracy). SHAP preferred; LIME is older but still used for non-tree models.

Q93. What is model drift?

[Medium]

Model performance degrades as real-world data distribution changes from training data. Fraud patterns shift monthly. Detect with KS tests on feature distributions weekly. Trigger retraining when recall < threshold.

Q94. What is the Kolmogorov-Smirnov test for drift?

[Hard]

Compares two distributions. `stats.ks_2samp(train_samples, prod_samples)`. `p_value < 0.05` = distributions statistically different = potential drift. Non-parametric: no assumption about distribution shape.

Q95. What artifacts should you log to MLflow?

[Hard]

Params: all hyperparams, threshold. Metrics: AUC-PR, recall, precision, savings. Artifacts: `model.pkl`, `confusion_matrix.png`, `pr_curve.png`, SHAP summary, `requirements.txt`, git commit hash.

Q96. What is GitHub Actions CI pipeline?

[Medium]

Automated workflow triggered on push/PR. Runs: checkout code, setup Python, pip install, lint (flake8), pytest. Returns green checkmark if all pass. Catches bugs before they reach main branch.

Q97. What is CD (Continuous Deployment)?

[Medium]

Automatically deploys passing builds to production or staging. In our workflow: build Docker image, push to Docker Hub, trigger Render redeployment. All automated on push to main.

Q98. Why test without real model files in CI?

[Hard]

`.pkl` files not committed to git (too large, DVC for data). Tests use `unittest.mock` to replace `model_store` with fake objects. CI can run tests without the model. Catches API logic errors.

Q99. What is pytest? How does it discover tests?

[Medium]

Python testing framework. Discovers functions starting with `'test_'` in files starting with `'test_'`. Run: `pytest tests/ -v`. Fixtures provide setup/teardown. `assert` statements verify expected behaviour.

Q100. What is a pytest fixture?

[Medium]

`@pytest.fixture` marks a function that provides test data/objects. pytest injects it into any test that lists it as a parameter. Used for: test client setup, mock model store, sample data.

Q101. Walk me through the full project end to end.

[Medium]

1.Problem: 0.17% fraud. 2.Data: 284k transactions, 30 features. 3.Scale Amount+Time. 4.Stratified 80/20 split. 5.SMOTE for LR, scale_pos_weight for XGBoost. 6.F2-threshold tuning. 7.Business cost analysis. 8.MLflow tracking. 9.FastAPI + Docker. 10.GitHub Actions CI/CD.

Q102. What was hardest technical challenge?

[Medium]

Imbalance and threshold selection. Default 0.5 threshold + unweighted training = 0% fraud recall. Required shift from model-centric (accuracy) to business-centric (money saved) thinking. F2-beta threshold was the key insight.

Q103. How is this different from a Kaggle project?

[Medium]

Kaggle: maximise leaderboard, ignore deployment. Production: FastAPI endpoint, drift monitoring, SHAP explanations, Docker, CI/CD, MLflow versioning, <100ms SLA, model rollback capability.

Q104. What is MLOps?

[Medium]

DevOps for ML. Covers: experiment tracking, CI/CD for models, model registry, monitoring, feature stores, A/B testing. Fills the gap between trained model and production-serving system.

Q105. How would you retrain the model when it degrades?

[Hard]

Trigger: recall < 80% or drift on 3+ features. Collect last 6 months confirmed labels. Run full pipeline. Validate: new model AUC-PR > old by 2%+. Shadow deploy 48h. Promote via MLflow registry.

Q106. What is a feature store?

[Hard]

Stores: offline (historical features for training), online (latest per-entity features, <10ms). Fraud features: velocity (tx per hour), amount deviation from customer mean. Tools: Feast, Tecton.

Q107. Describe scaling to Razorpay-scale (500M tx/day).

[Hard]

Training: distributed XGBoost on Spark. Features: Kafka streaming, Redis lookups. Inference: 5800 RPS average → 50-100 API pods. Model serving: Triton Inference Server for GPU batch. Storage: Parquet on S3.

Q108. What is graph-based fraud detection?

[Hard]

GNNs model transactions as graph: nodes=cards/merchants, edges=transactions. Fraud rings share merchant nodes or IPs. PyTorch Geometric, DGL. Detects collusive fraud that individual transaction models miss. Next level after tabular.

Q109. What is the business value? Put a number on it.

[Hard]

At scale: 500M tx/day, 0.1% fraud, avg £50 = £25M/day potential fraud. 85% recall catches £21.25M. FP blocking cost at 1% of legit = £10M friction. Net: £11.25M/day benefit. This framing wins business interviews.

Q110. When would you use a neural network instead of XGBoost?

[Hard]

100M+ transactions (NNs scale better), sequence models (LSTM for transaction sequences), multimodal (text + tabular), temporal patterns (attention). For <1M tabular: XGBoost wins almost always.

Q111. What is online vs batch learning?

[Hard]

Batch: train once on historical data, deploy static model, retrain periodically (this project). Online: model updates incrementally with each example. Adapts to new fraud patterns immediately but harder to debug.

Q112. What is active learning for fraud?

[Hard]

Model queries human analyst for labels on uncertain transactions. Fraud ops team labels borderline cases. Model retrains with new labels weekly. Handles new fraud types with few examples efficiently.

Q113. What is the difference between sensitivity and specificity?

[Medium]

Sensitivity = Recall = $TP/(TP+FN)$ = fraud caught rate. Specificity = $TN/(TN+FP)$ = legit correctly passed rate. Primary target: high sensitivity (catch fraud). Specificity should be high but secondary.

Q114. What is model calibration?

[Hard]

Well-calibrated model: when predicts $P(\text{fraud})=0.7$, 70% of those are actually fraud. XGBoost probabilities often not well-calibrated (push toward 0/1). Fix: `CalibratedClassifierCV` with isotonic regression.

Q115. What is lift in fraud context?

[Hard]

Lift = model's fraud capture in top N% / random capture rate. If top 1% of flagged transactions contains 50% of all fraud vs random 1% → lift = 50. Helps size fraud operations team.

Q116. What 2 more things would you add given 2 more weeks?

[Medium]

Week 1: velocity features (tx per hour per card), LightGBM comparison, proper 3-way split. Week 2: Kubernetes with HPA (auto-scaling), Grafana monitoring dashboard, graph fraud ring detection.

Q117. What is GDPR Article 22?

[Hard]

Right not to be subject to solely automated decisions with significant effects. Declined payment = significant effect. Banks must: explain logic (SHAP), allow human contest, provide human review. SHAP is legally required.

Q118. What is a model card?

[Hard]

Documentation: intended use, training data description, evaluation results, limitations, ethical considerations. For fraud: 'Designed for European credit card transactions, trained on 2013 data, may not generalise to crypto transactions.'

Q119. What is DVC (Data Version Control)?

[Hard]

Git for data and models. Tracks changes to large files stored in S3/GCS. Git tracks code, DVC tracks data. Each git commit can reference a specific dataset version. Enables full reproducibility.

Q120. What is the most important thing you learned?

[Easy]

Model metrics are not the product — business outcomes are. A deployed model with 70% AUC-PR that explains its decisions, monitors for drift, and saves £1M/month beats a 90% AUC-PR model in a notebook.

Category 5 — Quick Fire & Definitions (Q121–Q150)

Q121. What is precision?

[Easy]

$TP / (TP + FP)$. Of all fraud alerts, how many are real. High precision = few false alarms.

Q122. What is recall?

[Easy]

$TP / (TP + FN)$. Of all actual frauds, how many did we catch. High recall = few missed frauds.

Q123. What is F1 score?

[Easy]

Harmonic mean of precision and recall. $2 * P * R / (P + R)$. Equal weight on both. Use when FN and FP have equal cost.

Q124. What is AUC?

[Easy]

Area Under the Curve (ROC or PR). Threshold-independent metric. AUC-ROC=1.0 = perfect separation. AUC-PR preferred for imbalanced data.

Q125. What is overfitting vs underfitting?

[Easy]

Overfitting: great on train, poor on test (too complex). Underfitting: poor on both (too simple). Detect: train vs test metric gap.

Q126. What is cross-validation?

[Easy]

Train and evaluate on k different data splits. More reliable generalisation estimate. StratifiedKFold for imbalanced data.

Q127. What is a hyperparameter?

[Easy]

Value set before training (`n_estimators`, `learning_rate`). Not learned from data. Tuned via grid search, random search, or Optuna.

Q128. What is StandardScaler?

[Easy]

Transforms: $(x - \text{mean}) / \text{std}$. Result: mean=0, std=1. Fit on train only. Apply transform (not `fit_transform`) on test and API.

Q129. What is `predict_proba()`?

[Easy]

Returns probability array (`n_samples`, 2). Column 0 = P(legit), column 1 = P(fraud). Use `[:, 1]` for fraud probability. Needed for threshold tuning.

Q130. What is joblib?

[Easy]

Serialises Python objects to disk. Faster than pickle for numpy arrays. `joblib.dump(model, 'model.pkl')`, `joblib.load('model.pkl')`.

Q131. What is a REST API?

[Easy]

HTTP-based API. GET/POST/PUT/DELETE. Language-agnostic. FastAPI builds REST APIs with automatic validation and documentation.

Q132. What is JSON?

[Easy]

Text format for structured data. Language-agnostic. {key: value} pairs and lists. FastAPI auto-converts Python dicts to JSON.

Q133. What is Docker?

[Easy]

Packages app + dependencies + runtime in a container. Runs identically everywhere. docker build → docker run. Eliminates environment issues.

Q134. What is a Dockerfile?

[Easy]

Instructions to build a Docker image. FROM (base), COPY (files), RUN (commands), CMD (start command). Each line = one layer.

Q135. What is SMOTE stand for?

[Easy]

Synthetic Minority Oversampling Technique. Creates synthetic minority class samples by interpolating between real ones.

Q136. What is scale_pos_weight?

[Easy]

XGBoost parameter. Weight for positive class gradient = neg/pos count. Corrects class imbalance without synthetic data.

Q137. What is the classification threshold?

[Easy]

Cutoff: if $P(\text{fraud}) \geq \text{threshold}$, predict fraud. Default 0.5. Tune with PR curve. Lower = higher recall, lower precision.

Q138. What is MLflow?

[Easy]

Experiment tracking + model registry. Logs params, metrics, artifacts. Compare runs in UI. Version models in registry.

Q139. What is SHAP?

[Easy]

SHapley Additive exPlanations. Explains individual predictions. SHAP value = feature's contribution to pushing prediction above/below average.

Q140. What is GitHub Actions?

[Easy]

CI/CD built into GitHub. YAML workflows triggered on push/PR. Runs tests, builds Docker, deploys automatically.

Q141. What is Pydantic?

[Easy]

Python data validation library. BaseModel classes validate request/response schemas. Wrong types return 422 automatically.

Q142. What is uvicorn?

[Easy]

ASGI server that runs FastAPI. Single process. Use with Gunicorn for multi-process production: `gunicorn -w 4 -k uvicorn.workers.UvicornWorker`.

Q143. What is a confusion matrix?

[Easy]

2x2 table: TP, FP, FN, TN. `confusion_matrix(y_test, y_pred).ravel()` returns (TN, FP, FN, TP). Foundation of all classification metrics.

Q144. What is the PR curve?

[Medium]

Plot of Precision (y) vs Recall (x) at all thresholds. No-skill baseline = horizontal line at fraud rate (0.17%). Model must be above this.

Q145. What is early stopping?

[Medium]

XGBoost stops training when eval metric doesn't improve for N rounds. Prevents overfitting. Set `n_estimators` high, let `early_stopping_rounds` find optimal.

Q146. What is data leakage?

[Medium]

Test/future info influences training. Makes metrics unrealistically high in dev, fails in production. Prevent with proper split-first-then-transform order.

Q147. What is feature engineering?

[Medium]

Creating new features from raw data. Fraud examples: velocity (tx per hour), amount deviation from customer mean, time since last transaction, merchant risk score.

Q148. What is the lifespan pattern?

[Medium]

`asynccontextmanager` in FastAPI. Code before `yield` = startup. Code after `yield` = shutdown. Loads model once at startup into in-memory dict.

Q149. What is model drift?

[Medium]

Performance degrades as data distribution shifts from training data. Detect with KS tests. Trigger retraining when recall drops below target.

Q150. What makes this project production-ready?

[Medium]

End-to-end pipeline: data handling → imbalance correction → model training → threshold tuning → FastAPI endpoint → Docker → GitHub Actions CI/CD → MLflow tracking → SHAP explainability → drift monitoring.